

Resumo de Estudo — Aula 12

Forma Normal de Chomsky (FNC) • Seção 2.1

Já sabemos projetar gramáticas livres-do-contexto. Agora vamos aprender a colocá-las em uma **forma padronizada** — a Forma Normal de Chomsky — na qual todas as regras seguem um formato bem restrito. Essa padronização é a chave para criar algoritmos eficientes de análise sintática (como o CYK, da próxima aula) e para vários resultados teóricos.

1. O que é a Forma Normal de Chomsky

Ao trabalhar com GLCs é frequentemente conveniente tê-las numa forma simplificada. A mais simples e útil é a **Forma Normal de Chomsky (FNC)**. Suas utilidades principais: facilita projetar algoritmos de **análise sintática** (parsing) e serve de base para resultados de **computabilidade** (provar que toda LLC é decidível) e de **análise de complexidade**.

Definição 2.8 — Forma Normal de Chomsky

Uma GLC está na FNC se **todas** as suas regras têm um destes dois formatos (e mais duas condições sobre a variável inicial):

$V \rightarrow W X$	(duas variáveis)
$V \rightarrow a$	(um único terminal)

- Nas regras $V \rightarrow WX$, tanto W quanto X são **variáveis** (nunca terminais).
- Em $V \rightarrow a$, o lado direito é **um único terminal**.
- **(b)** A variável inicial S_0 não pode aparecer no lado **direito** de nenhuma regra.
- **(c)** Só a variável inicial pode ter a regra $S_0 \rightarrow \varepsilon$ (para permitir que a linguagem contenha a cadeia vazia).

*Em resumo: na FNC, cada passo de derivação ou junta duas variáveis, ou troca uma variável por um único terminal. Isso torna as árvores sintáticas **binárias**, o que é ótimo para algoritmos.*

2. Convertendo qualquer GLC para a FNC

Toda GLC pode ser transformada em uma equivalente na FNC (que gera exatamente a mesma linguagem). O procedimento tem **quatro passos**, e a cada passo a gramática continua equivalente à original:

Os quatro passos (nesta ordem)

- **Passo 1 — Nova variável inicial.** Adicione uma variável inicial nova S_0 e a regra $S_0 \rightarrow S$. Isso garante que a inicial nunca apareça à direita.
- **Passo 2 — Eliminar regras- ϵ** ($V \rightarrow \epsilon$, com $V \neq S_0$). Remova a regra e, para cada ocorrência de V no lado direito de outra regra, crie uma cópia da regra com essa ocorrência apagada.
- **Passo 3 — Eliminar regras unitárias** ($V \rightarrow U$, uma variável só). Remova a regra e copie para V todas as produções de U .
- **Passo 4 — Padronizar o restante** para os formatos $V \rightarrow WX$ ou $V \rightarrow a$, criando variáveis auxiliares onde necessário.

Passo 2 em detalhe (a parte mais delicada)

Ao eliminar $V \rightarrow \epsilon$: para **cada** ocorrência de V no lado direito de uma regra $U \rightarrow \dots V \dots$, adicione uma nova regra com essa ocorrência apagada. Se V aparece várias vezes na mesma regra, gere **todas** as combinações. Por exemplo, se $B \rightarrow uVvVw$, adicionam-se três regras:

$$B \rightarrow u \ v \ V \ w \quad | \quad u \ V \ v \ w \quad | \quad u \ v \ w$$

E se existir a regra $U \rightarrow V$ (só V à direita), adicione $U \rightarrow \epsilon$ — a menos que $U \rightarrow \epsilon$ já tenha sido removida antes (para não entrar em loop). A variável inicial S_0 é a única que pode manter $S_0 \rightarrow \epsilon$.

Passo 3 em detalhe

Ao eliminar a regra unitária $V \rightarrow U$: para toda regra $U \rightarrow u$, adicione $V \rightarrow u$ (a menos que seja uma unitária já removida). Regras da forma $V \rightarrow V$ são simplesmente descartadas (são redundantes). Repita até não sobrar nenhuma regra unitária.

Passo 4 em detalhe

Para regras longas $V \rightarrow u_1 u_2 \dots u_k$ com $k \geq 3$, quebre-as numa cadeia de regras binárias criando variáveis auxiliares $V_1, V_2, \dots$:

$$V \rightarrow u_1 \ V_1 \quad V_1 \rightarrow u_2 \ V_2 \quad V_2 \rightarrow u_3 \ V_3 \quad \dots \quad V_{(k-2)} \rightarrow u_{(k-1)} \ u_k$$

E qualquer terminal u_i que ainda apareça numa regra binária é substituído por uma nova variável U_i , com a regra $U_i \rightarrow u_i$.

3. Exemplo completo de conversão

Vamos converter a gramática abaixo passo a passo. Acompanhe como cada etapa modifica as regras — as versões intermediárias são G1, G2, ... até G8, a FNC final.

Gramática inicial G

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

Passo 1 — nova variável inicial (adiciona $S_0 \rightarrow S$):

G1

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

Passo 2 — eliminar $B \rightarrow \varepsilon$. B aparece em «aB» (vira «a») e em «A → B» (vira A → ε):

G2

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a$$
$$A \rightarrow B \mid S \mid \varepsilon$$
$$B \rightarrow b$$

Passo 2 (cont.) — agora eliminar a nova regra $A \rightarrow \varepsilon$. A aparece duas vezes em «ASA» (gerando SA, AS e S) — todas as combinações:

G3

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA \mid S$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b$$

(A regra $S \rightarrow S$ que surgiu será removida no passo seguinte, por ser unitária/redundante.)

Passo 3 — eliminar regras unitárias

Removemos $S \rightarrow S$ (redundante), depois $S_0 \rightarrow S$ (copiando as produções de S para S0), depois $A \rightarrow B$ (copiando b) e $A \rightarrow S$ (copiando as produções de S para A). O resultado após todas as eliminações unitárias:

G7

$S_0 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$
 $S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$
 $A \rightarrow b \mid ASA \mid aB \mid a \mid AS \mid SA$
 $B \rightarrow b$

Passo 4 — padronizar. As regras «ASA» (três símbolos) e «aB», «SA», «AS» (com terminal ou muito longas) são ajustadas criando variáveis auxiliares A1 (para SA) e A2 (para o terminal a):

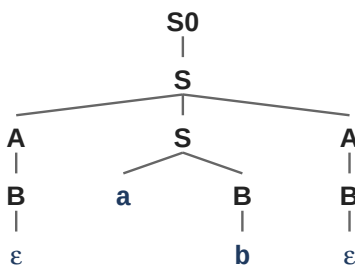
G8 (FNC final)

$S_0 \rightarrow AA1 \mid A2B \mid a \mid AS \mid SA$
 $S \rightarrow AA1 \mid A2B \mid a \mid AS \mid SA$
 $A \rightarrow b \mid AA1 \mid A2B \mid a \mid AS \mid SA$
 $A1 \rightarrow SA$
 $A2 \rightarrow a$
 $B \rightarrow b$

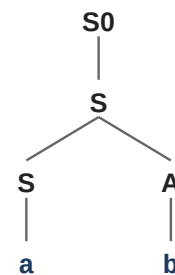
Todas as regras estão agora no formato $V \rightarrow WX$ ou $V \rightarrow a$. A gramática está na FNC. ✓

Por que a conversão preserva a linguagem?

A cada passo inserimos regras que "compensam" o que foi removido, de modo que as mesmas cadeias continuam sendo geradas. Veja a derivação da cadeia «ab» na gramática original G1 (à esquerda, com nós ϵ que serão eliminados) e na versão G3 (à direita, já sem os ϵ — a derivação simplesmente "pula" os passos que só produziam vazio):



Em G1: derivação de «ab» com nós ϵ



Em G3: mesma cadeia, sem os ϵ

4. Segundo exemplo de conversão

Convertendo a gramática de $L = \{ w \in \{a,b\}^* \mid w \text{ começa e termina com o mesmo símbolo} \}$ para a FNC. A gramática inicial (da Aula 11):

Gramática inicial

$$S \rightarrow S1 \mid S2$$
$$S1 \rightarrow a \mid aTa$$
$$S2 \rightarrow b \mid bTb$$
$$T \rightarrow Ta \mid Tb \mid \epsilon$$

Após adicionar $S0 \rightarrow S$, remover $T \rightarrow \epsilon$ (gerando aa , bb e $T \rightarrow a \mid b$), eliminar as unitárias (S , $S1$, $S2$ acabam desnecessárias) e padronizar as regras, chega-se a:

FNC final

$$S0 \rightarrow aA \mid AU \mid AA \mid bB \mid BV \mid BB$$
$$T \rightarrow TA \mid TB \mid a \mid b$$
$$U \rightarrow TA \qquad V \rightarrow TB$$
$$A \rightarrow a \qquad B \rightarrow b$$

(A regra $S0 \rightarrow aA$ vem de « aTa » após padronizar; U e V são auxiliares para « Ta » e « Tb »; A e B convertem os terminais a e b que apareciam junto de variáveis.)

5. Duas propriedades importantes da FNC

Essas propriedades são o que torna a FNC tão útil — e são a base do algoritmo CYK.

Propriedade 1 — Toda derivação tem exatamente $2n - 1$ passos

Dada uma gramática na FNC e uma cadeia $w \in L(G)$ de comprimento $n \geq 1$, a derivação de w usa **exatamente $2n - 1$ passos**.

Por quê: só há dois tipos de regra. As do tipo $V \rightarrow WX$ aumentam a cadeia intermediária em 1 símbolo — são necessárias $n - 1$ delas para chegar a n variáveis. As do tipo $V \rightarrow a$ apenas trocam variável por terminal (não mudam o tamanho) — são n delas. Total: $(n - 1) + n = 2n - 1$. Em termos de árvore: uma árvore binária com n folhas tem $n - 1$ nós internos, mais n substituições finais por terminais.

Consequência: dá para decidir se G gera w testando todas as derivações de $2n - 1$ passos. Isso *funciona*, mas tem custo **exponencial** — por isso precisamos de algo melhor (o CYK).

Propriedade 2 — Decomposição recursiva (base do CYK)

Se uma variável V deriva uma subcadeia $w_i \dots w_k$ (com $k > i$), então existe um ponto de quebra j ($i \leq j < k$) e duas variáveis V_1, V_2 tais que: G tem a regra $V \rightarrow V_1 V_2$, e $V_1 \Rightarrow^* w_i \dots w_j$ e $V_2 \Rightarrow^* w_{j+1} \dots w_k$.

Ou seja: para saber se V gera um pedaço da cadeia, basta **quebrá-lo em dois** e verificar se as duas metades são geradas por variáveis que se combinam por uma regra. Essa estrutura permite uma estratégia **bottom-up de tempo polinomial** — exatamente o algoritmo CYK da próxima aula.

Exemplo da propriedade 2: na gramática G_8 , temos $A \Rightarrow^* babab$ porque $A \rightarrow SA$, com $S \Rightarrow^* bab$ e $A \Rightarrow^* ab$. A subcadeia foi quebrada em duas partes (bab e ab), cada uma gerada por uma variável, e as duas se combinam pela regra $A \rightarrow SA$.

Exercícios resolvidos

Exercício 1 — Converta para a FNC: $S \rightarrow aSb \mid \varepsilon$

Passo 1 — nova inicial:

$$\begin{aligned} S_0 &\rightarrow S \\ S &\rightarrow aSb \mid \varepsilon \end{aligned}$$

Passo 2 — eliminar $S \rightarrow \varepsilon$. S aparece em «aSb» (vira «ab») e em « $S_0 \rightarrow S$ » (vira $S_0 \rightarrow \varepsilon$):

$$\begin{aligned} S_0 &\rightarrow S \mid \varepsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 3 — eliminar a unitária $S_0 \rightarrow S$ (copia as produções de S para S_0):

$$\begin{aligned} S_0 &\rightarrow aSb \mid ab \mid \varepsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 4 — padronizar. Criamos $A \rightarrow a$, $B \rightarrow b$, e quebramos «aSb» e «ab» em regras binárias ($X = Sb$; a auxiliar para o par):

$$\begin{aligned} S_0 &\rightarrow A X \mid A B \mid \varepsilon \\ S &\rightarrow A X \mid A B \\ X &\rightarrow S B \\ A &\rightarrow a \quad B \rightarrow b \end{aligned}$$

Todas as regras estão em $V \rightarrow WX$ ou $V \rightarrow a$, e só S_0 tem a regra $\rightarrow \varepsilon$. FNC completa. ✓

Exercício 2 — Por que a FNC é útil para algoritmos?

Porque as árvores sintáticas ficam **binárias** e toda derivação de uma cadeia de tamanho n tem exatamente $2n - 1$ passos (comprimento previsível). Combinada com a propriedade de decomposição recursiva (uma subcadeia é gerada por V sse puder ser quebrada em duas metades geradas por V_1 e V_2 com $V \rightarrow V_1 V_2$), isso permite preencher uma tabela de baixo para cima e decidir a pertinência em tempo polinomial $O(n^3)$ — é exatamente o que o algoritmo CYK faz.

Exercício 3 — Quantos passos tem a derivação de «aabb» numa GLC na FNC?

A cadeia «aabb» tem comprimento $n = 4$. Pela Propriedade 1, a derivação tem exatamente $2n - 1 = 2 \cdot 4 - 1 = 7$ **passos**: são $n - 1 = 3$ aplicações de regras $V \rightarrow WX$ (que montam a árvore binária de 4 folhas) e $n = 4$ aplicações de $V \rightarrow a$ (que colocam os terminais nas folhas). ✓